

Assignment 4 Code Review

Code Smell #1: Duplicate Code

- `WinMenu.java` and `GameOverMenu.java` have instances of duplicate code.
 - The timer row in both classes is nearly identical.
- Refactoring: Extracted the shared code into a static method `makeTimerRow(int timePlayed)` in the `Menu` base class. Both `WinMenu` and `GameOverMenu` now call this single method, so any future change only needs to be made in one place.

Code Smell #2: Long Methods

- The `generateBoard()` method in `BoardGenerator.java` is too long and could be divided into smaller methods to decrease size.
- Extracted `placeOuterWalls()` for floor filling and perimeter walls
- Extracted `placeRooms()` for server/storage room creation and entrance/exit placement
- Extracted `spawnAllEntities()` for entity spawning
- The new `generateBoard()` is now more understandable and high level.

Code Smell #3: Magic Numbers

- Hard-coded numeric literals in the code make it hard to understand its meaning.
- Occurs mainly in `BoardGenerator.java` in
 - Line 82: `int numShapes = 45;` (number of wall shapes)
 - Line 237: `for (int i = 0; i < 6; i++)` (number of data entities to spawn)
 - Line 240: `for (int i = 0; i < 10; i++)` (number of firewall entities to spawn)
- Refactoring: Extract them into named constants

```
private static final int TOTAL_FIREWALLS = 10;
private static final int TOTAL_DATA = 6;
private static final int WALL_SHAPES_COUNT = 45;
```

Code Smell #4: Long Method

- The `Hud` constructor was ~80 lines long, building the timer box, data box, banners, and freeze bar all inline with no separation of concerns. It was difficult to read and understand which part of the HUD each block was responsible for.
- Refactor: Extracted the timer box construction into `buildTimerBox(Runnable onPause)` and the data box construction into `buildDataBox()`. The constructor now reads as a clear high-level sequence of calls, with each helper method having a single focused responsibility.

Code Smell #5: Long Method

- The `placeWall()` method in `BoardGenerator.java` has too many responsibilities including boundary checking, server/storage room area validation, buffer zone validation, and wall placement logic.
- Refactor: extract validation logic into smaller helper methods like `isValidWallPosition()`, `isWithinBounds()`, `isInServer/StorageRoomArea()`, `isInBufferZone()`, `isInServer/StorageRoomBufferZone()`.

Code Smell #6: Long List of Parameter

- `buildBanner(String text, String iconAsset, String borderColor, double topAnchor, boolean isSprite)` had 5 parameters. The `iconAsset` string and `isSprite` boolean were two separate primitives that together just described one thing: which image to display.
- Replaced the `iconAsset + isSprite` pair with a single `Image` parameter. Callers now resolve the image themselves before calling `buildBanner`, reducing the parameter count to 4 and removing the boolean flag entirely.

Code Smell #7: Unjustified use of primitives

- The freeze progress bar animation used `Duration.millis(10000)`, a raw number with no explanation of where 10,000 came from. The actual freeze duration is defined in `FreezeToken.FREEZE_TIME` (100 ticks) and `VirusBreach.UPDATE_INTERVAL` (100ms per tick), but `Hud` ignored those and just duplicated the derived value as a magic primitive.
- Refactor: Introduced a named constant `FREEZE_DURATION_MS = FreezeToken.FREEZE_TIME * VirusBreach.UPDATE_INTERVAL` that computes the value from its source. The progress bar now references this constant, so if the freeze duration ever changes in the board logic, the UI automatically stays in sync.

Code Smell #8: Poor Package Structure

- `BoardGenerator` was placed in the package `ca.sfu.cmpt276.virusbreach` instead of in the package of classes that it directly communicated with, those being `Room`, `ServerRoom`, `StorageRoom`, and `Board`. This is misleading to onlookers and maybe confuse readers.
- It has been moved to `ca.sfu.cmpt276.virusbreach.board`. This new setup implies that it deals with board-level concerns rather than an application-level

concern, as indicated with being in the same package as classes like VirusBreach, GameTime, and GameOptions.